WatchWise

Interview Questions & Answers

52 technical + HR questions covering the full project: Django, models, face-recognition pipeline, OpenCV, threading, deployment, debugging and behavioral.

52 questions with answers

1. Project Overview

1. What is WatchWise, in one minute?

WatchWise is a Django web app that does real-time face recognition and attendance-style tracking across multiple cameras. People register with a webcam selfie, an admin authorizes them, and a background tracking engine recognizes them live on webcam or IP-camera feeds, logging check-in/check-out records on a monitoring dashboard.

2. Walk me through the end-to-end flow of the system.

Capture: a visitor registers with name, email, phone, class and a browser-webcam selfie, saved as a Person with authorized=False. Authorize: an admin toggles authorized=True. Track: the engine opens every configured camera in its own thread, detects faces with MTCNN, embeds them with InceptionResnetV1, matches them against authorized embeddings, draws green/red boxes, and creates a daily TrackingRecord with check-in, then check-out if the person is still visible after 60 seconds. Monitor: the browser shows live MJPEG feeds plus AJAX status panels.

3. What is the full tech stack?

Python 3.11, Django 5.0.7, SQLite, OpenCV 4.10, PyTorch 2.2 (CPU) + torchvision, facenet-pytorch 2.6 (MTCNN + InceptionResnetV1), NumPy 1.26, Pillow, pygame (alert sounds), HTML templates with AJAX polling. Dev tooling: uv virtualenv, Django migrations, runserver.

4. Why Django for this project instead of Flask or FastAPI?

Django gives auth, sessions, admin, ORM, migrations, forms and templating out of the box, so a single developer can ship a complete multi-page app with login, CRUD and dashboards fast. Flask/FastAPI would need those pieces assembled manually, which pays off only for pure APIs or microservices.

5. What was the hardest part of building WatchWise?

Real-time reliability: sharing frames safely between camera threads and HTTP streaming responses, keeping SQLite consistent under threaded writes with a lock, cutting latency on IP cameras with buffer-size-1 and frame skipping, and making detection robust in bad lighting with a 4-stage image-enhancement fallback.

2. Django Fundamentals

6. How does MVT map to WatchWise?

Models (app1/models.py: Person, TrackingRecord, CameraConfiguration) hold data and business rules. Views (app1/views.py) contain request handling plus the AI tracking engine. Templates (templates/) render home, monitor, person lists and camera forms. URLs (app1/urls.py, 23 routes) map paths to views.

7. What happens when you run 'python manage.py runserver'?

Django validates settings, runs system checks, and starts a single-threaded development WSGI server with an auto-reloader that restarts on code changes. It is for development only: no real concurrency, no static-file handling for production, insecure defaults.

8. Why function-based views here instead of class-based views?

Most views are simple render/redirect flows or custom logic (threads, streams) that do not fit generic CBVs like ListView. FBVs keep the AI-heavy custom code readable. The tradeoff is some repetition in the camera CRUD views, which CBVs could compress.

9. How does @login_required work, and what is LOGIN_URL?

The decorator checks request.user; anonymous users are redirected to LOGIN_URL ('login', resolving to /login/?next=<page>) and sent back after login. Views like monitor, ai-status and person_list use it so only authenticated users see cameras and people data.

10. What does @user_passes_test(is_admin) add on top?

A second gate requiring superuser status. Person detail/delete and all camera-configuration CRUD use it, so normal logged-in users can view the monitor but only admins can modify people or cameras.

11. What is the Django messages framework used for here?

One-shot user feedback across redirects, e.g. 'Person registered successfully', 'Tracking started', 'Camera configuration created'. Views push messages, templates display them once, then they are discarded.

12. Static files vs media files in this project?

Static files are app assets (CSS/JS); media files are user uploads served at MEDIA_URL from MEDIA_ROOT. Enrolled photos land in media/persons/ via ImageField(upload_to='persons/'), and dev serving is enabled with static() in WatchWise/urls.py.

3. Models and Database

13. Describe the three models and their relationships.

Person stores identity + photo + authorized flag. TrackingRecord has a ForeignKey to Person (many records per person, CASCADE delete) plus date, check_in_time, check_out_time. CameraConfiguration is standalone: camera name, source, recognition threshold, rotation setting.

14. Why is there db_index=True on TrackingRecord.person?

Almost every lookup filters by person (get_or_create per person per day, per-person history), so an index avoids full-table scans as records grow. Cheap to add, big win on read-heavy attendance queries.

15. Why get_or_create for the daily TrackingRecord?

It makes check-in idempotent: the first sighting of a person each day creates the row and stamps check-in; later sightings in the same thread loop fetch the same row instead of duplicating it. Without it, every processed frame could insert a duplicate.

16. What does select_related('person') do in tracking_record_list?

It JOINs Person in the same query so the template can show person names without one extra query per row (the N+1 problem). With hundreds of daily records this is the difference between 1 query and hundreds.

17. Why does TrackingRecord override save()?

To default date to today on creation only (if not self.pk). That guarantees one-record-per-person-per-day semantics even if callers forget to pass a date.

18. Why is a db_lock needed around database writes?

SQLite allows only one writer at a time and Django connections are per-thread. Multiple camera threads writing TrackingRecords concurrently would hit 'database is locked' errors, so a threading.Lock serializes all ORM writes.

19. How do migrations work in this project?

Change models.py, run makemigrations to generate a migration file (four exist, e.g. adding rotation to CameraConfiguration), then migrate to apply. The checked-in db.sqlite3 already has all four applied, and makemigrations --check confirms no pending changes.

4. Face Recognition and AI

20. Sketch the recognition pipeline in one breath.

Frame -> MTCNN finds face boxes + landmarks -> crop each face to 160x160 and normalize -> InceptionResnetV1 outputs a 512-number embedding -> Euclidean distance to each enrolled embedding -> closest match under threshold wins, else Not Recognized.

21. What is MTCNN and what are P-Net, R-Net, O-Net?

MTCNN is a cascaded face detector. P-Net proposes rough face candidates at multiple scales, R-Net rejects false ones and refines boxes, O-Net outputs the final box plus 5 facial landmarks (eyes, nose, mouth corners). Each stage has a strictness threshold; this project uses [0.5, 0.6, 0.6], tuned sensitive for low light.

22. What do keep_all=True and min_face_size=20 mean?

keep_all returns every detected face instead of only the largest, which is required for multi-person scenes. min_face_size=20 sets the smallest face the detector attempts; the code additionally drops boxes under 40x40 Fel because tiny crops give unreliable embeddings.

23. What is InceptionResnetV1 and what is it trained on?

A face-embedding network combining Inception modules with ResNet skip connections. The 'vggface2' weights were trained on ~3.3M photos of ~9,000 people using triplet loss, which pulls same-person photos together and pushes different people apart in vector space.

24. What exactly is the 512-dimensional embedding?

A numeric fingerprint of a face. Photos of the same person (different light, angle, expression) map to nearby vectors; different people map to distant ones. Identity is encoded, not pixels, which is why matching works across varying conditions.

25. What preprocessing happens before embedding?

Crop the detected box (clipped to image bounds), resize to 160x160, normalize with (pixel - 127.5) / 128.0, transpose to channel-first, add a batch dimension, and run under torch.no_grad() since no gradients are needed for inference.

26. How is a match decided? What is the 0.6 threshold?

numpy computes the Euclidean distance from the live embedding to every enrolled embedding and takes the minimum. Below the camera's threshold (default 0.6, the standard FaceNet operating point) it returns that person's name; otherwise 'Not Recognized'. Each camera has its own threshold because lighting differs per location.

27. Lower vs higher threshold: what breaks each way?

Too low (e.g. 0.4): strict, strangers never match, but known people get missed in bad light (false rejections). Too high (e.g. 0.8): lenient, recognizes in hard conditions, but risks matching strangers (false acceptances). Threshold is the security-vs-convenience dial.

28. What is the 4-stage detection fallback for dark frames?

If detection finds nothing: stage 1 retries on the original frame, stage 2 brightens dark frames (alpha=1.5, beta=30), stage 3 histogram-equalizes the Y channel, stage 4 sharpens with a kernel. Cheap classical preprocessing gives the neural net extra chances before giving up.

29. How are enrolled faces encoded and kept fresh?

encode_uploaded_images() runs the same MTCNN-to-ResNet pipeline over each authorized person's photo, caches embeddings for 5 minutes, and each tracking thread reloads every 2 minutes, so newly authorized people start being recognized without restarting the server.

5. OpenCV and Video

30. How are camera streams opened? Webcam vs IP camera?

If camera_source is all digits it opens cv2.VideoCapture(int(source)) as a local device index (0 = default webcam); otherwise the string is passed through as an RTSP/HTTP URL for IP cameras. Failure to open logs an error and the thread exits cleanly.

31. Why set CAP_PROP_BUFFERSIZE to 1 for IP cameras?

Network streams buffer several seconds of old frames by default, so the monitor lags reality. A buffer of 1 forces the freshest frame, trading occasional stutter for minimal latency, which matters for live recognition.

32. Why is the local webcam frame flipped?

cv2.flip(frame, 1) mirrors the local camera so it behaves like a mirror, matching user expectation on selfie-style feeds. IP cameras are not flipped since they are fixed surveillance views.

33. How does live video reach the browser?

Each thread JPEG-encodes its latest frame into a shared dict; video_feed returns a StreamingHttpResponse with content type multipart/x-mixed-replace, and the generator yields --frame boundaries forever, which browsers render as MJPEG video in an <img> tag.

34. Why keep both last_frames and last_frames_raw?

Processed frames have green/red boxes drawn for the monitor display; raw frames are clean captures used by single_frame for selfie registration, avoiding burned-in rectangles and CORS/stream-canvas issues.

6. Threading and Performance

35. Why one thread per camera instead of one loop?

Cameras are independent blocking I/O sources; a single loop reading them serially would stall all feeds on the slowest camera. One daemon thread per camera (spawned in start_tracking_internal) isolates failures and keeps every feed live.

36. How are tracking threads started and stopped safely?

Each thread gets a threading.Event in global_stop_events; stop_tracking sets all events, threads exit their while loops, release cv2 captures in a finally block, and frame dicts are cleared. Threads are daemon=True so a dead server never hangs on exit.

37. Why process only every 15th frame at half resolution?

MTCNN+ResNet on CPU cannot run at 30fps. Sharpening, downscaling 0.5x, and running AI every 15th frame (with boxes cached and upscaled between runs) keeps CPU usage sane while staying responsive enough for people walking past a camera.

38. What guards shared state between threads?

frame_lock protects the last_frames/last_frames_raw dicts read by streaming views while threads write them; db_lock serializes SQLite writes; last_recognized is a single-dict write which is atomic enough for a status display.

39. What did the port-8000 conflict teach about runserver?

Two Django projects bound the same port because of socket reuse, and requests landed on the wrong app (mystery 404s on valid routes). Lesson: check listeners and owning PIDs (Get-NetTCPConnection) before trusting a 404, and never assume your process owns the port.

7. Web Layer and UX

40. What are the AJAX endpoints and how is the monitor updated?

ai-status returns engine active + thread count; recognition-status returns the last recognized name if fresher than 5 seconds; authorize-toggle flips authorization. The monitor page polls them with JavaScript so status updates without full page reloads.

41. How does selfie registration work technically?

The browser captures a webcam frame to canvas, sends it as a base64 data URI in image_data, the view splits the header, base64-decodes it into a Django ContentFile, and saves a Person with authorized=False pending admin approval.

42. What is the authorization flow and why does it matter?

New people default to authorized=False and encode_uploaded_images() only embeds authorized people, so strangers can register but are never matched until an admin approves them. The authorization list page orders unauthorized first for quick review.

43. Explain the check-in/check-out logic precisely.

First sighting of a known person today: get_or_create makes the row and mark_checked_in stamps now plus a sound. On later sightings, if checked in but not out and 60+ seconds elapsed, mark_checked_out stamps now plus a sound. Same-day re-entry after checkout keeps the original row.

8. Deployment, DevOps and Debugging

44. How is WatchWise deployed today and what are the limits?

Locally via runserver on 127.0.0.1:8000 with SQLite and local media files. Limits: single-threaded dev server, no HTTPS, no process supervision, database and uploads live on one machine, secrets in settings.py, heavy torch/CPU load per machine.

45. How would you Dockerize WatchWise?

python:3.11-slim base, install system libs for OpenCV, pip install requirements (CPU torch), copy code, collectstatic, run gunicorn + a static server instead of runserver, mount volumes for db.sqlite3 and media/, expose 8000. One image then runs identically on any host.

46. How would you add CI/CD to this repo?

A GitHub Actions workflow on push: set up Python, install requirements, run manage.py check, run tests, optionally build the Docker image and push on main. Even this minimal pipeline catches broken imports and migration issues automatically.

47. Camera opens but returns no frames: how do you debug?

Checklist: verify the source string (index vs URL typo), test the RTSP/HTTP URL in VLC, confirm no other process holds the webcam (Windows locks devices: the code sleeps 0.5s before reopening for this), watch server logs for 'failed to return frame', and try buffer-size and timeout tweaks for network streams.

48. A registered person's photo never matches: what do you check?

Is the person authorized (only authorized faces are encoded)? Does the photo contain a detectable face (logs print 'No face found')? Is the file jpg/png/bmp and readable? Is the camera threshold too strict? Is the live lighting far worse than the enrollment photo? Force cache reload (encodings refresh every 2 minutes).

9. HR and Behavioral

49. Tell me about yourself.

I am a backend-leaning developer working in Python and Django. I built WatchWise, a real-time face-recognition attendance system with Django, OpenCV and PyTorch, which taught me to ship complete products: database design, auth, background threading, performance tuning and debugging. I am now deepening cloud and DevOps skills: Docker, CI/CD and AWS.

50. Why did you build this project?

I wanted to go beyond tutorials and integrate AI into a real working product: enrollment, authorization, live multi-camera tracking, records and alerting. Face recognition forced me to combine deep learning with very unglamorous engineering: threads, locks, latency and lighting.

51. What is the toughest bug you faced on WatchWise?

Valid routes returning 404: two Django projects were bound to port 8000 and requests landed on the wrong app. I traced it with port-to-PID inspection, killed the squatting process, and moved WatchWise to a clean port. It taught me to verify the running process before trusting the code.

52. How do you learn a new technology?

Build something real with it, read the official docs over tutorials, and debug the failures. Example: I learned CPU-only PyTorch deployment, uv venvs and Windows process inspection while getting this project's environment running from scratch.